

算法板子

区域赛现场速查版

统一从 `muban.cpp` 复制，按卡片接入

2026 年重构版

目录

第一章 使用说明、地基与索引	6
1.1 这本手册解决什么问题	6
1.2 统一代码地基	6
1.3 复杂度速查	7
1.4 关键词索引	7
1.5 卡片格式	8
第二章 基础算法与数据结构	9
2.1 前缀和与差分	9
2.2 并查集	10
2.3 树状数组	11
2.4 区间加、区间和线段树	11
2.5 二分答案	13
第三章 图论与树	14
3.1 统一建图约定	14
3.2 BFS、拓扑排序与 DAG DP	14
3.3 0-1 BFS	15
3.4 Dijkstra	16
3.5 Kruskal 最小生成树	17
3.6 SCC 与缩点	17
3.7 2-SAT	18
3.8 Dinic 最大流	19
3.9 最小费用最大流	21
3.10 LCA 倍增	22

第四章 字符串	23
4.1 KMP	23
4.2 Trie 与 AC 自动机	24
4.2.1 AC 自动机：分别统计每个模式串	26
4.3 后缀自动机 SAM	27
4.4 后缀数组与 LCP	29
4.5 回文自动机 PAM	30
第五章 数学、动态规划与几何	32
5.1 快速幂与组合数	32
5.2 扩展欧几里得与扩展 CRT	33
5.3 矩阵快速幂	34
5.4 数位 DP	34
5.5 SOS DP	35
5.6 Andrew 凸包	35
第六章 银牌档高频模板	37
6.1 莫队算法	37
6.2 可持久化线段树：区间第 k 小	38
6.3 重链剖分：树上路径和	39
6.4 树上启发式合并 DSU on Tree	40
6.5 单调斜率优化 CHT	42
第七章 错题附录：最小反例与提交前检查	43
7.1 提交前固定检查	43
7.2 最小反例表	44
7.3 对拍最小框架	44

1 使用说明、地基与索引

1.1 这本手册解决什么问题

这不是算法百科，而是区域赛现场的“关键词到代码”工具。看到题面后，先判断对象和约束，再翻到对应卡片。每张卡片都从上级目录的 `muban.cpp` 复制，随后追加该算法自己的结构和 `solve()` 接口。

现场流程：

1. 读清输入规模、是否多测、下标、取模和无解要求；
2. 用本章索引定位算法；
3. 复制 `muban.cpp`，再复制算法卡片中的追加代码；
4. 按“接入提示”改变量含义；
5. 先跑卡片样例，再跑边界测试。

1.2 统一代码地基

统一地基就是 `muban.cpp`，不再维护第二套头文件。当前版本的约定是：

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'
using i64 = long long;
using i128 = __int128_t;
// 别把这个改成具体的数，比赛的时候谁有时间给你重写
const int inf = 2e18+10;
const int N = 1e6 + 10;
const double eps = 1e-8;
```

本手册默认使用 `int`（宏展开为 `long long`），不处理需要 `unsigned` 的模板。算法卡片必须说明自己是否需要覆盖 `N`、`inf`、`eps`，以及新增的节点池大小。

#define int long long 后主函数写 signed main(); 位移写 1LL << k。数组下标仍然要检查范围，宏不会替代边界检查。

1.3 复杂度速查

规模	常见可接受复杂度	优先考虑
$n \leq 20$	$O(2^n n)$	状压、子集枚举
$n \leq 40$	$O(2^{n/2})$	折半搜索
$n \leq 200$	$O(n^3)$	Floyd、区间 DP、矩阵
$n \leq 2 \times 10^5$	$O(n \log n)$	排序、树状数组、线段树、Dijkstra
$n \leq 10^6$	$O(n)$ 或 $O(n \log \log n)$	扫描、哈希、线性筛
n 很大	$O(\log n)$ 或 $O(\log^2 n)$	快速幂、矩阵、数论分块

1.4 关键词索引

题面信号	先查
区间加、区间和	懒标记线段树
单点加、前缀和、第 k 小	树状数组 / 可持久化线段树
静态区间最值	稀疏表
离线区间统计	莫队
历史版本、版本差异	可持久化结构
无权最短路	BFS / 多源 BFS
边权只有 0, 1	0-1 BFS
非负权最短路	Dijkstra
负边或负环	Bellman-Ford / 差分约束
有向图缩点	SCC
真假约束、或关系	2-SAT
多模式串	AC 自动机
不同子串、出现次数	SAM
后缀排序、LCP	后缀数组
树上路径	LCA / HLD
子树颜色统计	DSU on Tree
树上距离在线查询	点分治 / 点分树
x^n 、线性递推	快速幂 / 矩阵快速幂
多个同余方程	CRT / 扩展 CRT
凸包、最远点树	Andrew / 旋转卡壳
矩形并面积	扫描线 + 线段树

1.5 卡片格式

每个算法在两本手册中都按以下顺序出现：

1. 识别关键词；
2. 适用条件和不能使用的情况；
3. 复杂度、内存、下标约定；
4. 状态或节点含义；
5. 追加代码；
6. 从 `muban.cpp` 接入的方式；
7. 完整样例输入输出；
8. 四组最小边界测试；
9. 常见错误和改造点。

2 基础算法与数据结构

2.1 前缀和与差分

难度：基础必会 关键词：区间静态求和、区间统一加

复杂度：预处理 $O(n)$ ，查询/修改 $O(1)$ 内存/前提：1-based 或 0-based 必须在卡片中统一

```
// pre[i] = a[1..i] 的和；下标 0 留作空前缀
vector<int> pre(n + 1, 0);
for (int i = 1; i <= n; ++i) pre[i] = pre[i - 1] + a[i];
auto range_sum = [&](int l, int r) {
    // 闭区间 [l,r] 的和 = 两个前缀相减
    return pre[r] - pre[l - 1];
};

// diff[l..r] 全部加 v，只需要修改两个端点
vector<int> diff(n + 2, 0);
auto add = [&](int l, int r, int v) {
    diff[l] += v;
    diff[r + 1] -= v;
};

for (int i = 1; i <= n; ++i) {
    // 对差分做前缀和，还原每个位置最终增加了多少
    diff[i] += diff[i - 1];
    a[i] += diff[i];
}
```

完整样例

输入：

```
4 2
1 2 3 4
1 3 5
2 4 -1
```

输出：

```
6 12 12 3
```

2.2 并查集

难度：基础必会 关键词：连通块、合并集合、最小生成树

复杂度：单次 $O(\alpha(n))$ 内存/前提：节点编号 1 到 n

```
struct DSU {  
    // fa 是父节点; sz 只在根节点上有意义, 用于按大小合并  
    vector<int> fa, sz;  
    explicit DSU(int n = 0) { init(n); }  
    void init(int n) {  
        fa.resize(n + 1);  
        sz.assign(n + 1, 1);  
        iota(fa.begin(), fa.end(), 0);  
    }  
    int find(int x) {  
        // 路径压缩: 查询后把 x 直接挂到根, 降低后续复杂度  
        return fa[x] == x ? x : fa[x] = find(fa[x]);  
    }  
    bool unite(int x, int y) {  
        x = find(x); y = find(y);  
        // 同根说明已经连通, 不能重复合并  
        if (x == y) return false;  
        // 小树挂到大树, 避免并查树退化  
        if (sz[x] < sz[y]) swap(x, y);  
        fa[y] = x; sz[x] += sz[y];  
        return true;  
    }  
};
```

样例:

输入:

```
5 4  
1 2  
2 3  
4 5  
1 3
```

输出:

```
3
```


表示最后有 3 个连通块。

2.3 树状数组

难度：现场常用 关键词：单点加、前缀和、逆序对、离散化后计数

复杂度：单次 $O(\log n)$ 内存/前提：下标从 1 开始

```
struct Fenwick {
    // bit[x] 保存以 x 结尾、长度 lowbit(x) 的区间和
    int n;
    vector<int> bit;
    explicit Fenwick(int n = 0) { init(n); }
    void init(int n_) { n = n_; bit.assign(n + 1, 0); }
    void add(int x, int v) {
        // x += lowbit(x): 更新所有覆盖位置 x 的节点
        for (; x <= n; x += x & -x) bit[x] += v;
    }
    int sum(int x) const {
        int res = 0;
        // x -= lowbit(x): 拆成互不重叠的若干块
        for (; x > 0; x -= x & -x) res += bit[x];
        return res;
    }
    int range_sum(int l, int r) const {
        return sum(r) - sum(l - 1);
    }
};
```

树状数组不能直接维护任意区间最值。若要区间加、单点查，把加法作用在差分数组上；若要区间加、区间和，需要两棵树或改用线段树。

2.4 区间加、区间和线段树

难度：现场常用 关键词：区间加、区间和、懒标记

复杂度：建树 $O(n)$ ，操作 $O(\log n)$ 内存/前提：节点池按 $4n$ 开

```
struct SegTree {
    // sum[p] 是节点 p 对应区间的总和，lazy[p] 是尚未下传的区间加
    int n;
    vector<int> sum, lazy;
```

```

explicit SegTree(int n = 0) { init(n); }
void init(int n_) {
    n = n_;
    sum.assign(4 * n + 5, 0);
    lazy.assign(4 * n + 5, 0);
}
void apply(int p, int l, int r, int v) {
    // 整段加 v: 总和增加 v * 区间长度, 标记累加
    sum[p] += v * (r - l + 1);
    lazy[p] += v;
}
void push(int p, int l, int r) {
    // 叶子没有子节点; 没有标记也无需下传
    if (!lazy[p] || l == r) return;
    int mid = (l + r) >> 1;
    apply(p << 1, l, mid, lazy[p]);
    apply(p << 1 | 1, mid + 1, r, lazy[p]);
    lazy[p] = 0;
}
void add(int p, int l, int r, int ql, int qr, int v) {
    // 当前区间被完全覆盖时, 打标记后立即返回
    if (ql <= l && r <= qr) return apply(p, l, r, v);
    // 部分覆盖前必须先把父标记推给孩子
    push(p, l, r);
    int mid = (l + r) >> 1;
    if (ql <= mid) add(p << 1, l, mid, ql, qr, v);
    if (qr > mid) add(p << 1 | 1, mid + 1, r, ql, qr, v);
    sum[p] = sum[p << 1] + sum[p << 1 | 1];
}
int query(int p, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return sum[p];
    push(p, l, r);
    int mid = (l + r) >> 1, res = 0;
    if (ql <= mid) res += query(p << 1, l, mid, ql, qr);
    if (qr > mid) res += query(p << 1 | 1, mid + 1, r, ql, qr);
    return res;
}
};

```

从 ‘muban.cpp’ 复制后, 先把初始数组逐点调用 ‘add(1,1,n,i,i,a[i])’, 再接题目操作。每次访问子区间前必须 ‘push’。

2.5 二分答案

难度：基础必会 关键词：最小的最大值、最大的最小值、可行性判定

复杂度： $O(n \log V)$ 内存/前提：先确认判定函数单调

```
int lo = *max_element(a.begin(), a.end()); // 答案下界：至少容纳单个最大值
int hi = accumulate(a.begin(), a.end(), 0LL); // 答案上界：全部元素放一组
while (lo < hi) {
    int mid = (lo + hi) >> 1;
    // check(mid) 必须是单调函数：可行则尝试更小，否则只能增大
    if (check(mid)) hi = mid;
    else lo = mid + 1;
}
cout << lo << endl;
```

二分答案不是看到“最小/最大”就能套。必须先证明‘check(x)’的真假随 x 单调变化，且‘lo’、‘hi’覆盖真实答案。

3 图论与树

3.1 统一建图约定

除非题目明确要求，否则本手册的图编号从 1 开始。无向边加入两次；有向边只加入一次。多测时重建 ‘vector<vector<Edge>>’ 或清空边池。

3.2 BFS、拓扑排序与 DAG DP

难度：基础必会 关键词：无权最短路、依赖顺序、DAG 上转移

复杂度： $O(n + m)$ 内存/前提：拓扑序长度小于 n 表示有环

```
vector<int> bfs_dist(int s, const vector<vector<int>>> &g) {
    // d[v] = 从 s 到 v 的最少边数；-1 表示尚未到达
    vector<int> d(g.size(), -1);
    queue<int> q; q.push(s); d[s] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) if (d[v] == -1)
            // 第一次到达 v 时，当前路径一定最短
            d[v] = d[u] + 1, q.push(v);
    }
    return d;
}

vector<int> topo_sort(const vector<vector<int>>> &g, vector<int> indeg) {
    queue<int> q;
    // 入度为 0 的点可以作为当前拓扑序的下一个点
    for (int i = 1; i < (int)g.size(); ++i)
        if (!indeg[i]) q.push(i);
    vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop(); order.push_back(u);
        // 删除 u 的出边；新变成 0 入度的点入队
        for (int v : g[u]) if (--indeg[v] == 0) q.push(v);
    }
    return order;
}
```

```
}
```

完整样例

输入边： $1 \rightarrow 2, 1 \rightarrow 3, 2 \rightarrow 4, 3 \rightarrow 4$ 。

BFS 从 1 的距离为 ‘0 1 1 2’；拓扑序可以是 ‘1 2 3 4’ 或 ‘1 3 2 4’。加入 $4 \rightarrow 1$ 后，拓扑序长度变小，说明有环。

3.3 0-1 BFS

难度：现场常用 **关键词：**边权只有 0 和 1、最短路

复杂度： $O(n + m)$ **内存/前提：**0 边入队首，1 边入队尾

```
struct Edge01 { int to, w; };
vector<vector<Edge01>> g;
vector<int> dist;

void bfs01(int s) {
    // deque 保证队首总是当前最有希望的距离
    deque<int> q;
    dist.assign(g.size(), inf);
    dist[s] = 0;
    q.push_front(s);
    while (!q.empty()) {
        int u = q.front(); q.pop_front();
        for (auto e : g[u]) {
            if (dist[e.to] > dist[u] + e.w) {
                dist[e.to] = dist[u] + e.w;
                // 权 0 不增加距离，应优先处理；权 1 放到队尾
                if (e.w == 0) q.push_front(e.to);
                else q.push_back(e.to);
            }
        }
    }
}
```

样例

输入：

```
3 3
1 2 0
2 3 1
1 3 0
```

从 1 到 3 的最短距离为 '0'。若出现权值 2，不能继续使用这份模板。

3.4 Dijkstra

难度：基础必会 关键词：非负边权最短路

复杂度： $O((n+m)\log n)$ 内存/前提：不能有负边

```
struct Edge { int to, w; };
vector<vector<Edge>> g;
vector<int> dist;

void dijkstra(int s) {
    // 只适用于所有边权非负的图
    dist.assign(g.size(), inf);
    priority_queue<pair<int,int>,
                  vector<pair<int,int>>,
                  greater<pair<int,int>>> pq;
    dist[s] = 0;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        // 同一个点可能多次入堆；旧距离不是当前最优值时跳过
        if (d != dist[u]) continue;
        for (auto e : g[u]) {
            if (dist[e.to] > d + e.w) {
                dist[e.to] = d + e.w;
                pq.push({dist[e.to], e.to});
            }
        }
    }
}
```

堆中允许同一个点出现多次，必须跳过旧距离。不要把 Dijkstra 用到存在负边的图上；负边先查 Bellman-Ford 或差分约束。

完整样例

输入：

```
3 3 1
1 2 5
1 3 1
3 2 1
```

输出距离：‘0 2 1’。点 2 的旧堆项距离 5 必须被跳过。

3.5 Kruskal 最小生成树

难度：基础必会 关键词：连接所有点、最小总边权

复杂度： $O(m \log m)$ 内存/前提：最后选边数必须为 $n - 1$

```
struct E { int u, v, w; };
int kruskal(int n, vector<E> edges) {
    // 按边权从小到大尝试，只选连接两个不同连通块的边
    sort(edges.begin(), edges.end(),
        [](const E& a, const E& b) { return a.w < b.w; });
    DSU dsu(n);
    int ans = 0, used = 0; // used 是已选边数，用来判断图是否连通
    for (auto e : edges) {
        if (!dsu.unite(e.u, e.v)) continue;
        ans += e.w;
        if (++used == n - 1) break;
    }
    return used == n - 1 ? ans : -1;
}
```

3.6 SCC 与缩点

难度：现场常用 关键词：有向图互相可达、缩成 DAG

复杂度： $O(n + m)$ 内存/前提：Kosaraju 需要正反图

```
vector<vector<int>> g, rg;
vector<int> vis, order, comp;

void dfs1(int u) {
    // 第一遍在原图上求后序；结束时间大的点优先进入第二遍
    vis[u] = 1;
    for (int v : g[u]) if (!vis[v]) dfs1(v);
    order.push_back(u);
}

void dfs2(int u, int c) {
    // 反图上的一次 DFS 恰好得到一个强连通分量
    comp[u] = c;
    for (int v : rg[u]) if (comp[v] == -1) dfs2(v, c);
}

int scc(int n) {
```

```

vis.assign(n + 1, 0);
comp.assign(n + 1, -1);
order.clear();
for (int i = 1; i <= n; ++i) if (!vis[i]) dfs1(i);
reverse(order.begin(), order.end());
int cnt = 0;
for (int u : order) if (comp[u] == -1) dfs2(u, cnt++);
return cnt;
}

```

样例

输入边： $1 \rightarrow 2, 2 \rightarrow 1, 2 \rightarrow 3, 3 \rightarrow 4, 4 \rightarrow 3$ 。

输出分量： $\{1, 2\}, \{3, 4\}$ 。缩点后只保留 $\{1, 2\} \rightarrow \{3, 4\}$ 。

3.7 2-SAT

难度：现场常用 **关键词：**变量真假、子句“ $x \vee y$ ”、互斥选择

复杂度： $O(n + m)$ **内存/前提：**每个变量拆成真假两个点

```

int id(int x, int val) { return x * 2 + val; }
// val=0 表示 false, val=1 表示 true。
vector<vector<int>> sat_g, sat_rg;
vector<int> sat_vis, sat_order, sat_comp;
void init_2sat(int n) {
    sat_g.assign(2 * n, {});
    sat_rg.assign(2 * n, {});
}
void sat_dfs1(int u) {
    sat_vis[u] = 1;
    for (int v : sat_g[u]) if (!sat_vis[v]) sat_dfs1(v);
    sat_order.push_back(u);
}
void sat_dfs2(int u, int c) {
    sat_comp[u] = c;
    for (int v : sat_rg[u]) if (sat_comp[v] == -1) sat_dfs2(v, c);
}
void add_or(int x, int a, int y, int b) {
    // (x=a) or (y=b) 等价于 (!x=a)→(y=b) 和 (!y=b)→(x=a)
    // 子句 (x=a) or (y=b) 的两个蕴含边：
    // not(x=a) → (y=b), not(y=b) → (x=a)
    int u1 = id(x, a ^ 1), v1 = id(y, b);
    int u2 = id(y, b ^ 1), v2 = id(x, a);
    sat_g[u1].push_back(v1); sat_rg[v1].push_back(u1);
}

```



```

    sat_g[u2].push_back(v2); sat_rg[v2].push_back(u2);
}
bool solve_2sat(int n) {
    int V = 2 * n;
    sat_vis.assign(V, 0);
    sat_comp.assign(V, -1);
    sat_order.clear();
    for (int i = 0; i < V; ++i) if (!sat_vis[i]) sat_dfs1(i);
    reverse(sat_order.begin(), sat_order.end());
    int c = 0;
    for (int u : sat_order)
        if (sat_comp[u] == -1) sat_dfs2(u, c++);
    // x 与 not x 在同一 SCC 时，互相推出矛盾，不存在解
    for (int i = 0; i < n; ++i)
        if (sat_comp[id(i, 0)] == sat_comp[id(i, 1)]) return false;
    return true;
}

```

最容易错的是蕴含边方向。先写逻辑等价式，再按“假设一边为假，必须推出另一边为真”加边。

完整样例接口：先调用 `init_2sat(n)`；每行 `x a y b` 表示 $(x = a) \vee (y = b)$ ，其中变量编号从 0 开始、‘a,b’ 为 0/1。

输入：

```

2 3
0 0 1 0
0 1 1 0
0 0 1 1

```

输出：‘YES’。若加入 ‘0 0 0 1’ 和 ‘0 1 0 1’ 两个互相矛盾的子句，输出 ‘NO’。

3.8 Dinic 最大流

难度：现场常用 **关键词：**容量、割、匹配、选取与冲突建模

复杂度：常见 $O(EV^2)$ **内存/前提：**反向边容量必须存在

```

struct FlowEdge { int to, rev, cap; };
struct Dinic {
    // rev 是反向边在对方邻接表中的下标，修改残量时必须同步两条边
    int n;
    vector<vector<FlowEdge>> g;
    vector<int> level, it;

```

```
explicit Dinic(int n): n(n), g(n), level(n), it(n) {}

void add_edge(int u, int v, int cap) {
    FlowEdge a{v, (int)g[v].size(), cap};
    FlowEdge b{u, (int)g[u].size(), 0};
    g[u].push_back(a); g[v].push_back(b);
}

bool bfs(int s, int t) {
    // level 图只允许沿层数 +1 的边增广
    fill(level.begin(), level.end(), -1);
    queue<int> q; q.push(s); level[s] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (auto &e : g[u])
            if (e.cap && level[e.to] == -1)
                level[e.to] = level[u] + 1, q.push(e.to);
    }
    return level[t] != -1;
}

int dfs(int u, int t, int f) {
    if (u == t) return f;
    // it[u] 记住当前弧，避免重复扫描已经失败的边
    for (int &i = it[u]; i < (int)g[u].size(); ++i) {
        auto &e = g[u][i];
        if (e.cap && level[e.to] == level[u] + 1) {
            int got = dfs(e.to, t, min(f, e.cap));
            if (got) {
                e.cap -= got;
                g[e.to][e.rev].cap += got;
                return got;
            }
        }
    }
    return 0;
}

int max_flow(int s, int t) {
    int ans = 0, f;
    // 每轮建立一张 level 图，并在其上尽可能增广
    while (bfs(s, t)) {
        fill(it.begin(), it.end(), 0);
        while ((f = dfs(s, t, inf)) != 0) ans += f;
    }
    return ans;
}

};
```

3.9 最小费用最大流

难度：高风险模板 关键词：运输、带费用匹配、要求最小代价

复杂度：每次增广一次最短路 内存/前提：先确认负费用边和需求不足的输出规则

```
struct CostEdge { int to, rev, cap, cost; };
vector<vector<CostEdge>> cg;
void add_cost_edge(int u, int v, int cap, int cost) {
    CostEdge a{v, (int)cg[v].size(), cap, cost};
    CostEdge b{u, (int)cg[u].size(), 0, -cost};
    cg[u].push_back(a); cg[v].push_back(b);
}
pair<int,int> min_cost_flow(int s, int t, int need) {
    int flow = 0, cost = 0, n = cg.size();
    while (flow < need) {
        // SPFA 在残量图中寻找一条费用最小的增广路
        vector<int> d(n, inf), pv(n), pe(n);
        vector<char> inq(n, false);
        queue<int> q; q.push(s); d[s] = 0; inq[s] = true;
        while (!q.empty()) {
            int u = q.front(); q.pop(); inq[u] = false;
            for (int i = 0; i < (int)cg[u].size(); ++i) {
                auto &e = cg[u][i];
                if (e.cap && d[e.to] > d[u] + e.cost) {
                    d[e.to] = d[u] + e.cost;
                    pv[e.to] = u; pe[e.to] = i;
                    if (!inq[e.to]) inq[e.to] = true, q.push(e.to);
                }
            }
        }
        if (d[t] == inf) break;
        // 汇点不可达：剩余网络无法继续送流
        int add = need - flow;
        for (int v = t; v != s; v = pv[v])
            add = min(add, cg[pv[v]][pe[v]].cap);
        for (int v = t; v != s; v = pv[v]) {
            auto &e = cg[pv[v]][pe[v]];
            e.cap -= add; cg[v][e.rev].cap += add;
        }
        flow += add; cost += add * d[t];
    }
    return {flow, cost};
}
```

完整样例

唯一通路容量为 3、单位费用为 5，要求流量 2，输出 ‘(2,10)’。

总容量只有 1 而要求流量 2 时，输出流量必须是 1，不能伪造满流。

3.10 LCA 倍增

难度：基础必会 **关键词：**树上路径、最近公共祖先、树上差分

复杂度：预处理 $O(n \log n)$ ，查询 $O(\log n)$ **内存/前提：**树必须连通或逐个连通块处理

```
const int LOG = 21;
vector<vector<int>> tree;
vector<array<int, LOG>> up;
vector<int> depth;

void dfs_lca(int u, int p) {
    // up[u][j] = u 向上跳 2^j 步到达的祖先
    up[u][0] = p;
    for (int j = 1; j < LOG; ++j)
        up[u][j] = up[up[u][j - 1]][j - 1];
    for (int v : tree[u]) if (v != p) {
        depth[v] = depth[u] + 1;
        dfs_lca(v, u);
    }
}

int lca(int u, int v) {
    // 先把较深的点抬到同一深度
    if (depth[u] < depth[v]) swap(u, v);
    int d = depth[u] - depth[v];
    for (int j = 0; j < LOG; ++j) if (d >> j & 1) u = up[u][j];
    if (u == v) return u;
    for (int j = LOG - 1; j >= 0; --j)
        if (up[u][j] != up[v][j])
            u = up[u][j], v = up[v][j];
    return up[u][0];
}
```

‘LOG’ 要覆盖最大深度；根节点的父亲必须设为自己或 0，并保持整棵树的初始化一致。

4 字符串

4.1 KMP

难度：基础必会 关键词：单模式匹配、重叠出现、周期

复杂度： $O(n + m)$ 内存/前提：模式串为空时按题面单独处理

```
vector<int> prefix_function(const string &p) {
    // pi[i]: p[0..i] 的最长真前后缀长度
    vector<int> pi(p.size());
    for (int i = 1; i < (int)p.size(); ++i) {
        int j = pi[i - 1];
        // 失配时沿着已经求出的 border 链跳，不回退 i
        while (j && p[i] != p[j]) j = pi[j - 1];
        if (p[i] == p[j]) ++j;
        pi[i] = j;
    }
    return pi;
}

vector<int> kmp(const string &text, const string &pat) {
    if (pat.empty()) return {};
    auto pi = prefix_function(pat);
    vector<int> pos;
    for (int i = 0, j = 0; i < (int)text.size(); ++i) {
        // j 表示当前已经匹配的 pat 前缀长度
        while (j && text[i] != pat[j]) j = pi[j - 1];
        if (text[i] == pat[j]) ++j;
        if (j == (int)pat.size()) {
            pos.push_back(i - j + 1);
            j = pi[j - 1]; // 保留重叠前缀
        }
    }
    return pos;
}
```

样例

输入：文本 ‘abababa’，模式 ‘aba’。

输出：匹配起点 ‘0 2 4’，数量为 ‘3’。

‘aaaaa’ 与 ‘aaa’ 的起点是 ‘0 1 2’，用来检查匹配成功后不能把 ‘j’ 直接清零。

4.2 Trie 与 AC 自动机

难度：现场常用 **关键词：**多个模式串、文本扫描、后缀模式

复杂度：建树 $O(S\Sigma)$ ，扫描 $O(|T|)$ **内存/前提：**模式总长度为 S ，只支持小写字母

下面这份代码的 ‘out[u]’ 是“以节点 u 结尾的模式串数量”。“build()” 后把缺失转移补齐，因此只能在插入完成后调用一次；多测必须重新构造对象。

```
struct AC {
    // go 是 Trie 转移; fail[u] 指向 u 的最长真后缀节点
    // terminal[u] 只统计恰好在 u 结束的模式串（允许重复）
    // output[u] 是沿 fail 链可匹配的模式串总数
    vector<array<int, 26>> go;
    vector<int> fail, terminal, output;

    AC() { init(); }
    void init() {
        go.clear(); fail.clear();
        terminal.clear(); output.clear();
        go.push_back({}); go[0].fill(0);
        fail.push_back(0);
        terminal.push_back(0);
        output.push_back(0);
    }
    int insert(const string &s) {
        int u = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (!go[u][c]) {
                // 首次经过这条字符边时创建 Trie 节点
                go[u][c] = go.size();
                go.push_back({});
                go.back().fill(0);
                fail.push_back(0);
                terminal.push_back(0);
                output.push_back(0);
            }
            u = go[u][c];
        }
    }
};
```

```

    }
    ++terminal[u];
    return u;
}

void build() {
    queue<int> q;
    // 根的 fail 为 0; 第一层节点直接入队
    for (int c = 0; c < 26; ++c) {
        if (go[0][c]) q.push(go[0][c]);
    }
    while (!q.empty()) {
        int u = q.front(); q.pop();
        // output 汇总当前节点以及 fail 链上的终止模式串
        output[u] = terminal[u] + output[fail[u]];
        for (int c = 0; c < 26; ++c) {
            int &v = go[u][c];
            if (v) {
                // 已存在的 Trie 边: 由父节点的 fail 转移计算 fail[v]
                fail[v] = go[fail[u]][c];
                q.push(v);
            } else {
                // 补全自动机转移, 查询时无需 while 回退
                v = go[fail[u]][c];
            }
        }
    }
    output[0] = terminal[0];
}

int query_total(const string &text) const {
    int u = 0, ans = 0;
    for (char ch : text) {
        // 每个字符只走一次自动机边, 累计当前位置结尾的所有模式
        u = go[u][ch - 'a'];
        ans += output[u];
    }
    return ans;
}
};

```

完整样例

模式串: 'a'、'ab'、'bab'; 文本: 'abab'。

每个模式串出现次数分别为:

$a : 2, \quad ab : 2, \quad bab : 1.$

如果需要分别统计，把 ‘insert()’ 返回的结束节点保存下来，扫描后在 fail 树上汇总文本经过次数；不能只读取 Trie 节点本身。

‘output[u]’ 版本适合统计文本中所有模式串的总出现次数。若题目要求每个模式串分别输出，必须另写 fail 树统计版本，不能把两种语义混用。

4.2.1 AC 自动机：分别统计每个模式串

仍然复用上一个 AC 对象。扫描文本时给当前状态计数，再沿 fail 树从深到浅汇总；因此“模式串互为后缀”时，短模式也会收到长模式经过节点的贡献。返回的 `pattern_node[i]` 是第 i 个模式串插入结束的节点，重复模式串可以保留多个相同节点。

```
vector<int> count_each_pattern(const AC &ac, const string &text,
                             const vector<int> &pattern_node) {
    vector<int> pass(ac.go.size(), 0), order;
    int u = 0;
    for (char ch : text) {
        // build() 已补齐转移，所以扫描阶段不需要沿 fail while 回退。
        u = ac.go[u][ch - 'a'];
        ++pass[u];
    }

    // fail 边反向后是一棵树；反向遍历保证先汇总深层节点。
    vector<vector<int>> fail_tree(ac.go.size());
    for (int v = 1; v < (int)ac.go.size(); ++v)
        fail_tree[ac.fail[v]].push_back(v);
    vector<int> st = {0};
    while (!st.empty()) {
        int x = st.back(); st.pop_back();
        order.push_back(x);
        for (int v : fail_tree[x]) st.push_back(v);
    }
    reverse(order.begin(), order.end());
    for (int x : order)
        if (x) pass[ac.fail[x]] += pass[x];

    vector<int> answer;
    answer.reserve(pattern_node.size());
    for (int node : pattern_node) answer.push_back(pass[node]);
    return answer;
}
```


完整输入输出：

模式串：a, ab, bab

文本：abab

每个模式串出现次数：2 2 1

四组边界：模式集合为空；重复模式 ‘a,a’；后缀模式 ‘a,ba,b a’（去掉空格后为 ‘a,ba,ba’）；文本为空。多测时直接重新构造 AC，不要在旧对象上再次 build()。

4.3 后缀自动机 SAM

难度：高风险模板 **关键词：**不同子串、子串出现次数、最长重复子串

复杂度：构建 $O(n)$ ，状态数 $O(n)$ **内存/前提：**字符串长度决定节点池；每组数据必须 init

```
struct SAM {
    struct Node {
        // go: 从该状态出发的字符转移; len: 该状态能表示的最长串长度
        // link: 后缀链接; occ: 在线插入时该状态被作为前缀经过的次数
        array<int, 26> go{};
        int link = 0, len = 0, occ = 0;
    };
    vector<Node> st;
    int last;

    void init(int n) {
        // SAM 最多 2n-1 个状态; 1 号状态作为根, 0 号位置不用
        st.assign(2 * n + 5, Node{});
        st[1] = Node{};
        last = 1;
        st.resize(2);
    }

    void extend(int c) {
        // 新建 cur, 代表把字符 c 接到当前整串末尾
        int cur = st.size();
        st.push_back(Node{});
        st[cur].len = st[last].len + 1;
        st[cur].occ = 1;
        int p = last;
        while (p && !st[p].go[c]) {
            // 沿后缀链接补上所有缺失的 c 转移
            st[p].go[c] = cur;
            p = st[p].link;
        }
        if (!p) st[cur].link = 1;
```

```

else {
    int q = st[p].go[c];
    if (st[p].len + 1 == st[q].len) st[cur].link = q;
    else {
        int clone = st.size();
        // q 的长度过长时复制出 clone, 保持每个状态的长度区间合法
        st.push_back(st[q]);
        st[clone].len = st[p].len + 1;
        st[clone].occ = 0;
        while (p && st[p].go[c] == q) {
            st[p].go[c] = clone;
            p = st[p].link;
        }
        st[q].link = st[cur].link = clone;
    }
}
last = cur;
}

void count_occurrence() {
    // 按 len 降序把出现次数从子状态汇总到后缀链接父状态
    vector<int> ord(st.size() - 1);
    iota(ord.begin(), ord.end(), 1);
    sort(ord.begin(), ord.end(),
        [&](int a, int b) { return st[a].len > st[b].len; });
    for (int u : ord)
        if (st[u].link) st[st[u].link].occ += st[u].occ;
}
};

```

完整题例：P3804 的输入是一行字符串，要求所有子串中 长度 $\times$ 出现次数 的最大值。把每个字符调用一次 `extend(s[i]-'a')`，再调用 `count_occurrence()`，输出 $\max_{u \neq 1} \text{len}[u] * \text{occ}[u]$ 。

输入：ababa

输出：6

四组边界：空串（答案 0）、‘a’（1）、‘aaaa’（6）、‘ab’（2）。若改问不同子串数量，答案是 $\sum_{u \neq 1} (\text{len}[u] - \text{len}[\text{link}[u]])$ ，不需要汇总 `occ`。每组数据必须重新调用 `init(n)`。

4.4 后缀数组与 LCP

难度：高风险模板 关键词：后缀排序、LCP、不同子串

复杂度：排序版 $O(n \log^2 n)$ 内存/前提：工程上先用排序版，数据很大再换基数排序

```
vector<int> suffix_array(string s) {
    int n = s.size();
    if (n == 0) return {};
    vector<int> sa(n), rk(n), tmp(n);
    iota(sa.begin(), sa.end(), 0);
    // rk[i] 是当前按前 2k 个字符比较时，后缀 i 的等价类编号
    for (int i = 0; i < n; ++i) rk[i] = s[i];
    for (int k = 1;; k <= 1) {
        // 每轮按（前半段排名，后半段排名）排序，倍增比较长度
        sort(sa.begin(), sa.end(), [&](int i, int j) {
            if (rk[i] != rk[j]) return rk[i] < rk[j];
            int ri = i + k < n ? rk[i + k] : -1;
            int rj = j + k < n ? rk[j + k] : -1;
            return ri < rj;
        });
        tmp[sa[0]] = 0;
        for (int i = 1; i < n; ++i) {
            int a = sa[i - 1], b = sa[i];
            int a2 = a + k < n ? rk[a + k] : -1;
            int b2 = b + k < n ? rk[b + k] : -1;
            tmp[b] = tmp[a] + (rk[a] != rk[b] || a2 != b2);
        }
        rk.swap(tmp);
        // 所有后缀排名都不同，排序完成
        if (rk[sa.back()] == n - 1) break;
    }
    return sa;
}
```

```
vector<int> kasai_lcp(const string &s, const vector<int> &sa) {
    int n = s.size();
    vector<int> rank(n), lcp(n);
    for (int i = 0; i < n; ++i) rank[sa[i]] = i;
    int h = 0;
    for (int i = 0; i < n; ++i) {
        int r = rank[i];
        if (r == 0) continue;
        int j = sa[r - 1];
        // h 是上一位置的 LCP，向右移动后最多只需减一。
    }
```

```

    while (i + h < n && j + h < n && s[i + h] == s[j + h]) ++h;
    lcp[r] = h;
    if (h) --h;
}
return lcp; // lcp[r] = LCP(sa[r-1], sa[r]), lcp[0] = 0
}

```

完整题例：字符串 ‘banana’ 的后缀数组为 ‘5 3 1 0 4 2’，`kasai_lcp` 返回 ‘0 1 3 0 0 2’。lcp[r] 的下标跟后缀数组排名一致，任意两个后缀的 LCP 可继续在 lcp 上做 RMQ。

四组边界：空串；‘a’；‘aaaa’（大量相同前缀）；‘ab’（无公共前缀）。排序版复杂度为 $O(n \log^2 n)$ ，若 n 很大再替换排序器，不要在现场混用两版下标。

4.5 回文自动机 PAM

难度：高风险模板 **关键词：**不同回文子串、最长回文后缀、出现次数

复杂度：构建 $O(n)$ ，节点数 $\leq n + 2$ **内存/前提：**只支持小写字母；每组数据调用 `init`

每个节点代表一个不同的回文串。0 号根长度为 -1 ，1 号根是空串；`fail[u]` 指向去掉两端后最长的回文后缀。插入字符后，`last` 是当前前缀的最长回文后缀。

```

struct PAM {
    struct Node {
        array<int, 26> go{};
        int fail = 0, len = 0;
        i64 occ = 0; // 先统计最长回文后缀次数，再沿 fail 汇总
    };
    vector<Node> tr;
    vector<int> text;
    int last = 1, n = 0;

    void init(int max_len) {
        tr.clear(); tr.reserve(max_len + 2);
        text.clear(); text.reserve(max_len);
        tr.push_back(Node{}); tr.push_back(Node{});
        tr[0].len = -1; tr[0].fail = 0; // -1 根
        tr[1].len = 0; tr[1].fail = 0; // 空串根
        last = 1; n = 0;
    }

    int get_fail(int u) const {
        while (n - 1 - tr[u].len < 0 ||
               text[n - 1 - tr[u].len] != text[n])
            u = tr[u].fail;
        return u;
    }
}

```

```
}
int add(char ch) {
    int c = ch - 'a';
    text.push_back(c);
    int cur = get_fail(last);
    if (!tr[cur].go[c]) {
        int now = tr[cur].go[c] = tr.size();
        tr.push_back(Node{});
        tr[now].len = tr[cur].len + 2;
        tr[now].fail = (tr[now].len == 1)
            ? 1 : tr[get_fail(tr[cur].fail)].go[c];
    }
    last = tr[cur].go[c];
    ++tr[last].occ;
    ++n;
    return last;
}
void count_occurrence() {
    vector<int> ord(tr.size() - 2);
    iota(ord.begin(), ord.end(), 2);
    sort(ord.begin(), ord.end(),
        [&](int a, int b) { return tr[a].len > tr[b].len; });
    for (int u : ord) tr[tr[u].fail].occ += tr[u].occ;
}
};
```

完整题例：插入 'abacaba' 后，tr.size()-2 为 7，表示 'a,b,c,aba,bac,cab,abacaba' 等不同非空回文子串的数量。输入 'a'、'aaaa'、'abacaba'、'ababa' 时输出分别为 '1,4,7,5'。如果题目问出现次数，先调用 count_occurrence()；如果只问不同回文串数量，直接输出 tr.size()-2。多测必须重新 init(max_len)。

5 数学、动态规划与几何

5.1 快速幂与组合数

难度：基础必会 关键词：幂、组合数、取模

复杂度：快速幂 $O(\log b)$ ，组合数预处理 $O(n)$ 内存/前提：模数为质数且 $n < \text{mod}$ 时使用阶乘逆元

```
int qpow(int a, int b, int mod) {
    // res 保存已经处理的二进制位贡献；a 每轮平方
    int res = 1 % mod;
    a %= mod;
    while (b) {
        // 当前最低位为 1，说明这一位的幂需要乘进答案
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

struct Comb {
    // fac[i]=i!, ifac[i]=(i!)^{-1}, 均按 mod 取模
    int mod;
    vector<int> fac, ifac;
    void init(int n, int p) {
        mod = p;
        fac.assign(n + 1, 1);
        ifac.assign(n + 1, 1);
        for (int i = 1; i <= n; ++i) fac[i] = fac[i - 1] * i % mod;
        ifac[n] = qpow(fac[n], mod - 2, mod);
        // 费马小定理求 n! 的逆元，再递推所有较小逆阶乘
        for (int i = n; i; --i) ifac[i - 1] = ifac[i] * i % mod;
    }
    int C(int n, int k) const {
        if (k < 0 || k > n) return 0;
```

```

        return fac[n] * ifac[k] % mod * ifac[n - k] % mod;
    }
};

```

模数不是质数时不能直接使用费马逆元；组合数下标超过模数时先查 Lucas 或非质数模数模板。

5.2 扩展欧几里得与扩展 CRT

难度：现场常用 **关键词：**多个同余式、模数不互质、求最小非负解

复杂度：每次合并 $O(\log m)$ **内存/前提：**无解时必须返回标志

```

int exgcd(int a, int b, int &x, int &y) {
    // 返回 gcd(a,b), 并求出 a*x+b*y=gcd(a,b)
    if (!b) { x = 1; y = 0; return a; }
    int x0, y0;
    int g = exgcd(b, a % b, x0, y0);
    x = y0;
    y = x0 - (a / b) * y0;
    return g;
}

// 合并 x = a1 (mod m1), x = a2 (mod m2)
bool merge_crt(int a1, int m1, int a2, int m2, int &a, int &m) {
    int x, y, g = exgcd(m1, m2, x, y);
    int d = a2 - a1;
    // 两式可合并的充要条件：差值能被 gcd(m1,m2) 整除
    if (d % g) return false;
    int lcm = m1 / g * m2;
    int k = (1LL * d / g * x % (m2 / g));
    if (k < 0) k += m2 / g;
    a = (a1 + (1LL * m1 * k) % lcm);
    if (a < 0) a += lcm;
    m = lcm;
    return true;
}

```

样例

$x \equiv 1 \pmod{4}$, $x \equiv 3 \pmod{6}$ 的最小非负解为 $9 \pmod{12}$ 。

$x \equiv 0 \pmod{2}$, $x \equiv 1 \pmod{4}$ 无解，必须输出题目规定的无解标志。

5.3 矩阵快速幂

难度：现场常用 关键词：线性递推、固定维度状态转移

复杂度： $O(k^3 \log n)$ 内存/前提：矩阵维度必须很小

```
struct Mat {
    // a[i][j] 表示从状态 j 对第 i 个新状态的线性贡献
    int n;
    vector<vector<int>> a;
    Mat(int n = 0, bool identity = false): n(n), a(n, vector<int>(n)) {
        if (identity) for (int i = 0; i < n; ++i) a[i][i] = 1;
    }
};

Mat operator*(const Mat &A, const Mat &B) {
    Mat C(A.n);
    // 普通矩阵乘法; i128 防止中间乘法溢出 long long
    for (int i = 0; i < A.n; ++i)
        for (int k = 0; k < A.n; ++k) if (A.a[i][k])
            for (int j = 0; j < A.n; ++j)
                C.a[i][j] = (C.a[i][j] +
                    (i128)A.a[i][k] * B.a[k][j]) % 1000000007;
    return C;
}

Mat mpow(Mat A, int e) {
    // R 初始为单位矩阵, 按 e 的二进制位做快速幂
    Mat R(A.n, true);
    while (e) {
        if (e & 1) R = R * A;
        A = A * A; e >>= 1;
    }
    return R;
}
```

5.4 数位 DP

难度：现场常用 关键词：统计 $[0, R]$ 中满足数字限制的数

复杂度：状态数乘位数 内存/前提：前导零、tight 和 $L - 1$ 是三大边界

```
string s;
// pos=当前位, tight=是否仍贴着上界, started=是否已经出现非前导零
int memo[20][2][2];
bool seen[20][2][2];
```



```
int dfs(int pos, int tight, int started) {
    // 到末尾代表构造出一个合法数字；这里把“0”也计入
    if (pos == (int)s.size()) return 1;
    if (seen[pos][tight][started]) return memo[pos][tight][started];
    seen[pos][tight][started] = true;
    int digit = s[pos] - '0';
    int lim = tight ? digit : 9, ans = 0;
    for (int d = 0; d <= lim; ++d) {
        if (d == 4) continue;
        // 只有仍贴着上界且 d 等于当前上界位时，下一层才 tight
        ans += dfs(pos + 1, tight && d == digit, started || d != 0);
    }
    return memo[pos][tight][started] = ans;
}
```

正式代码中 ‘tight && d == lim’ 应写成与当前上界数字比较，而不是与固定 9 比较。统计 $[L, R]$ 时使用 ‘F(R)-F(L-1)’，并单独处理 $L = 0$ 。

5.5 SOS DP

难度：现场常用 关键词：子集和、超集和、位掩码统计
复杂度： $O(n2^n)$ 内存/前提：掩码位数通常不超过 20

```
// f[mask] 初始为恰好 mask 的值；逐位把子集贡献累加到超集
for (int bit = 0; bit < n; ++bit)
    for (int mask = 0; mask < (1LL << n); ++mask)
        if (mask & (1LL << bit))
            f[mask] += f[mask ^ (1LL << bit)]; // 子集和
```

5.6 Andrew 凸包

难度：现场常用 关键词：凸包、极值点、旋转卡壳前处理
复杂度：排序 $O(n \log n)$ 内存/前提：重复点先去重；保留边界点时改叉积条件

```
struct Point {
    // 使用 long double 保存坐标，eps 来自 muban.cpp
    long double x, y;
    bool operator<(const Point &o) const {
        return x != o.x ? x < o.x : y < o.y;
    }
};
```

```
    }
};

long double cross(Point a, Point b, Point c) {
    // (b-a) x (c-a): >0 左转, <0 右转, 约等于 0 共线
    return (b.x - a.x) * (c.y - a.y)
        - (b.y - a.y) * (c.x - a.x);
}

vector<Point> convex_hull(vector<Point> p) {
    // 先排序去重, 再分别构造下凸壳和上凸壳
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end(), [](Point a, Point b) {
        return fabs1(a.x - b.x) < eps && fabs1(a.y - b.y) < eps;
    }), p.end());
    if (p.size() <= 1) return p;
    vector<Point> h;
    for (auto x : p) {
        // 右转或共线时弹出中间点, 保留外层边界
        while (h.size() >= 2 &&
            cross(h[h.size()-2], h.back(), x) <= eps) h.pop_back();
        h.push_back(x);
    }
    int lower = h.size();
    // lower 记录下凸壳结束位置, 之后追加下凸壳
    for (int i = (int)p.size() - 2; i >= 0; --i) {
        while ((int)h.size() > lower &&
            cross(h[h.size()-2], h.back(), p[i]) <= eps) h.pop_back();
        h.push_back(p[i]);
    }
    h.pop_back();
    return h;
}
```

6 银牌档高频模板

本章只收录区域赛中值得提前准备、但不适合临场重新推导的模板。每个模板都必须在复制后用本节样例和边界测试跑过。

6.1 莫队算法

难度：现场常用 **关键词：**离线区间统计，移动左右端点的代价小

复杂度：排序 $O((n+q)\sqrt{n})$ **内存/前提：**不能处理在线询问；必须实现对称的 add/remove

```
struct MoQuery { int l, r, id; };
int block_size;
vector<int> a, freq; // a 是离散化后的数组, freq[x] 是当前窗口中 x 的次数
int distinct_count;

void add_pos(int p) {
    // 把位置 p 纳入窗口; 第一次出现时不同数个数 +1
    if (++freq[a[p]] == 1) ++distinct_count;
}

void remove_pos(int p) {
    // 移除位置 p; 最后一次消失时不同数个数 -1
    if (--freq[a[p]] == 0) --distinct_count;
}

vector<int> mo_distinct(vector<MoQuery> qs, int n) {
    // 莫队把查询排序, 使窗口左右端点移动总量尽量小
    block_size = max<int>(1, sqrt(n));
    sort(qs.begin(), qs.end(), [&](auto x, auto y) {
        int bx = x.l / block_size, by = y.l / block_size;
        if (bx != by) return bx < by;
        return (bx & 1) ? x.r > y.r : x.r < y.r;
    });
    freq.assign(n + 1, 0);
    distinct_count = 0;
    vector<int> ans(qs.size());
    int l = 0, r = -1; // 初始窗口为空, 维护闭区间 [l,r]
    for (auto q : qs) {
```

```

        while (l > q.l) add_pos(--l);
        while (r < q.r) add_pos(++r);
        while (l < q.l) remove_pos(l++);
        while (r > q.r) remove_pos(r--);
        ans[q.id] = distinct_count;
    }
    return ans;
}

```

完整样例

```

5 3
1 2 1 3 2
0 2
1 4
0 4

```

输出：

```

2
3
3

```

这里询问使用 0-based 闭区间；数组离散化后再维护 ‘freq’。第四组边界测试使用空区间、单点区间和全部相同数组。

6.2 可持久化线段树：区间第 k 小

难度：现场常用 **关键词：**历史版本、静态区间第 k 小

复杂度：单次查询 $O(\log n)$ **内存/前提：**值域先离散化；每个版本只复制访问路径

```

struct PNode { int l = 0, r = 0, sum = 0; };
vector<PNode> pst(1); // 0 号空节点，未走到的子树都指向它
vector<int> roots, values;

int update(int old, int L, int R, int pos) {
    // 新版本复制旧版本的路径，其余节点与旧版本共享
    int cur = pst.size();
    pst.push_back(pst[old]);
    ++pst[cur].sum; // 当前值域区间内出现次数 +1
    if (L != R) {
        int M = (L + R) >> 1;
        if (pos <= M) pst[cur].l = update(pst[old].l, L, M, pos);
        else pst[cur].r = update(pst[old].r, M + 1, R, pos);
    }
}

```

```

    }
    return cur;
}
int kth(int left_root, int right_root, int L, int R, int k) {
    // 两个前缀版本相减，得到原数组区间 [l,r] 的频次
    if (L == R) return L;
    int M = (L + R) >> 1;
    int left_count = pst[pst[right_root].l].sum
        - pst[pst[left_root].l].sum;
    // 左半边数量足够就向左，否则扣掉左边数量向右找
    if (k <= left_count)
        return kth(pst[left_root].l, pst[right_root].l, L, M, k);
    return kth(pst[left_root].r, pst[right_root].r, M + 1, R,
        k - left_count);
}

```

完整样例

```

5 2
5 1 4 4 2
2 5 2
2 5 3

```

输出：

```

2
4

```

查询前要把离散化排名映射回原值；‘k’ 不在区间长度内时按题面处理无解。

6.3 重链剖分：树上路径和

难度：现场常用 **关键词：**树上路径修改/查询、子树操作

复杂度：预处理 $O(n)$ ，单次 $O(\log^2 n)$ **内存/前提：**DFS 序连续覆盖子树；路径拆成若干重链

```

vector<vector<int>> tree;
vector<int> parent, depth, sz, heavy, top, dfn, rev;
int timer;

void dfs1(int u, int p) {
    // 第一次 DFS 求子树大小，并找每个点最大的重儿子
    parent[u] = p; sz[u] = 1; heavy[u] = -1;
    int best = 0;

```

```

    for (int v : tree[u]) if (v != p) {
        depth[v] = depth[u] + 1;
        dfs1(v, u); sz[u] += sz[v];
        if (sz[v] > best) best = sz[v], heavy[u] = v;
    }
}

void dfs2(int u, int tp) {
    // 第二次 DFS 给重链连续编号; top[u] 是当前重链顶端
    top[u] = tp; dfn[u] = ++timer; rev[timer] = u;
    if (heavy[u] != -1) dfs2(heavy[u], tp);
    for (int v : tree[u])
        if (v != parent[u] && v != heavy[u]) dfs2(v, v);
}

int path_sum(int u, int v) {
    int ans = 0;
    while (top[u] != top[v]) {
        // 不在同一条重链时, 先查询深度更大的链顶并跳到其父亲
        if (depth[top[u]] < depth[top[v]]) swap(u, v);
        ans += seg.query(1, 1, n, dfn[top[u]], dfn[u]);
        u = parent[top[u]];
    }
    if (depth[u] > depth[v]) swap(u, v);
    return ans + seg.query(1, 1, n, dfn[u], dfn[v]);
}

```

这段路径函数依赖基础章节的线段树对象 ‘seg’。接入时先把点权按 ‘dfn’ 顺序建入线段树；边权问题要把边权挂到更深的端点，并在查询时排除 LCA。

完整样例

树边：‘1-2, 1-3, 3-4, 3-5’，点权全为 1。查询路径 ‘(2,5)’ 的和为 ‘3’；把点 3 加 4 后，再查询输出 ‘7’。

6.4 树上启发式合并 DSU on Tree

难度：高风险模板 **关键词：**每个子树颜色频次、众数和

复杂度：通常 $O(n \log n)$ **内存/前提：**先求重儿子；轻子树统计后必须清空

```

vector<vector<int>> tree;
vector<int> color, sz, heavy, ans, cnt;
int big_child, best_freq, best_sum;

void dfs_size(int u, int p) {

```

```

    sz[u] = 1; heavy[u] = -1;
    int best = 0;
    for (int v : tree[u]) if (v != p) {
        dfs_size(v, u); sz[u] += sz[v];
        if (sz[v] > best) best = sz[v], heavy[u] = v;
    }
}

void add_subtree(int u, int p, int delta) {
    // delta=+1 加入子树, delta=-1 清空子树
    cnt[color[u]] += delta;
    if (cnt[color[u]] > best_freq ||
        (cnt[color[u]] == best_freq && color[u] < best_sum))
        best_freq = cnt[color[u]], best_sum = color[u];
    for (int v : tree[u])
        if (v != p && v != big_child) add_subtree(v, u, delta);
}

void dfs_sack(int u, int p, bool keep) {
    // 轻儿子处理后清空, 重儿子保留, 保证总复杂度
    for (int v : tree[u])
        if (v != p && v != heavy[u]) dfs_sack(v, u, false);
    if (heavy[u] != -1) dfs_sack(heavy[u], u, true), big_child = heavy[u];
    add_subtree(u, p, 1);
    big_child = -1;
    ans[u] = best_sum;
    if (!keep) {
        add_subtree(u, p, -1);
        best_freq = best_sum = 0;
    }
}
}

```

真实题目若要求“最大频率颜色之和”，答案维护器要改成维护所有达到最大频率的颜色和；上面只展示“频率最高且取最小颜色”的示意接口，复制前必须按题意修改。

完整样例

树边：‘1-2,1-3,3-4,3-5’，颜色为‘1,2,2,3,2’。

每个子树中出现次数最多的颜色（取最小编号）输出：‘1 2 2 3 2’。

6.5 单调斜率优化 CHT

难度：高风险模板 关键词：DP 转移可化为直线、斜率和查询单调

复杂度：均摊 $O(1)$ 内存/前提：只有在斜率插入顺序和查询顺序满足单调性时使用

```
struct Line { int k, b; int value(int x) const { return k * x + b; } };
deque<Line> hull;
bool bad(Line a, Line b, Line c) {
    // b 在几何上被 a、c 遮挡时永远不会成为最优线
    return (i128)(b.b - a.b) * (b.k - c.k)
        >= (i128)(c.b - b.b) * (a.k - b.k);
}
void add_line(Line x) {
    while (hull.size() >= 2 &&
        bad(hull[hull.size()-2], hull.back(), x)) hull.pop_back();
    hull.push_back(x);
}
int query(int x) {
    // 查询点 x 单调递增时，队首劣线可以永久弹出
    while (hull.size() >= 2 &&
        hull[0].value(x) >= hull[1].value(x)) hull.pop_front();
    return hull.front().value(x);
}
```

如果斜率或查询点不单调，不能直接使用这份 deque 版 CHT，应改用李超线段树；本重构不收录李超树的未验证变体。

完整样例

依次加入直线 $y = 1x + 0$ 、 $y = 2x - 1$ ，按 $x = 0, 1, 2$ 查询最大值，输出 ‘0 1 3’。如果题目的斜率插入顺序或查询顺序不单调，不能套这份代码。

7 错题附录：最小反例与提交前检查

这一章只保留能在比赛中直接排错的信息。每条错误都使用“最小输入—错误原因—修复动作”的格式，不再写长篇复盘。

7.1 提交前固定检查

1. 是否读对了测试组数、边数、字符串和空行；
2. 所有乘法是否可能超过 `long long`，中间量是否需要 `i128`；
3. `inf + w` 是否会溢出；
4. 模数是否为质数，除法是否真的可以使用逆元；
5. 下标是 0-based 还是 1-based，区间是闭区间还是半开区间；
6. 多测时图、节点池、时间戳、全局答案和懒标记是否重建；
7. 是否测试了 $n = 0, 1, 2$ 、全相同、全负数、无解、不连通、重边和自环；
8. 输出大小写、顺序、精度、无解标志是否符合题面。

7.2 最小反例表

模块	最小反例	主要检查
二分	数组 <code>[1]</code> ，答案在端点	初始边界和返回位置
差分	区间 <code>[1,n]</code> 加一次	<code>'r+1'</code> 是否越界
线段树	整段加后查询单点	是否下传懒标记
并查集	重复合并同一点	是否错误增加连通数
Dijkstra	旧堆项 + 不可达点	是否判断 <code>'d != dist[u]'</code>
0-1 BFS	一条 0 边和一条 1 边	0 边是否进队首
MST	不连通图	是否检查选边数 $n - 1$
桥	两条平行边	是否按边编号跳过父边
KMP	<code>'aaaaa'</code> 与 <code>'aaa'</code>	是否保留重叠前缀
AC	<code>'a,ab,bab'</code> 与 <code>'abab'</code>	fail 后缀统计
SAM	<code>'aaaa'</code> 、 <code>'ab'</code>	clone 和初始化
DP	单个负数 <code>[-5]</code>	答案是否错误初始化为 0
几何	重合点、相切圆	eps 和退化分支

7.3 对拍最小框架

```
for (int seed = 1; ; ++seed) {
    // gen 产生合法小数据
    // brute 输出正确答案
    // sol 输出待测答案
    // 比较不同后立即保存输入并停止
}
```

错题卡只记录五项：

1. 题号和模板；
2. 最小错误输入；
3. 第一个违反的不变量；
4. 修改的关键行；
5. 回归样例和正确输出。